EPFL - Computer Science

CS-401 Applied Data Analysis

Lecture 10 — 19/11/2025

Author: Jonathan Dutoit

Professor: Maria Brbic

Handling text data (Part 1)

Textual Data

- Much modern data is unstructured text
  - Web
  - Social media
  - News
  - Several ADA project datasets...
- Frequently, ”clean” datasets can be derived from ”dirty” textual data:
  - e.g., search queries are short texts; Google Trends time series for concepts are obtained by aggregating all search queries referring to the concept.

Nov 2022: The Dawn of a New Era: because of LLMs (ChatGPT).

Outline

- 4 typical tasks on text data:
  - Document retrieval
  - Document classification
  - Sentiment analysis
  - Topic detection
- How to phrase these tasks as machine learning problems
- How to pre-process text so it can be fed to ML algorithms
- ADA spirit: show you what's there; give you basic feel

Document Retrieval

Typical task 1

- **Given**
  - Document collections (a.k.a corpus)
  - Query document (can be short query string)
- **Task**
  - Rank all docs in collection by similarly to query
- An old problem (e.g., libraries)
- Document retrieval is the core task solved by Web search engines ("10 blue links")

Document retrieval

- Straightforward approach: neighbor search (as in kNN)
- Define a distance function between documents
- Given query q, find the k docks with smallest distance to q
- $k = 10$, docs sorted by distance, blue links, ads $\Rightarrow$ Google
- The hard part: craft/learn a distance function (and scale it to the Web...)

Document classification

Typical task 2

- **Given:**
  - Document d
  - Set of classes (e.g., topics: news, sports, tech, music, romance)
- **Task:**
  - Decide to which one of the classes document d belongs
- Example scenario:
  - Find gangster movies in CMU Movie Summary Corpus

Document classifications

- Supervised learning
- Obtain a large collection of documents
- Label each doc with the class it belongs to
- Represent docs as feature vectors
- Train a supervised classifier based on the labeled docs:

  - e.g., kNN, logistic regression, decision tree, random forest, boosted decision trees, neural network, ...

Sentiment Analysis

Typical task 3

- **Given:**
  - Document d (e.g., product review)
- **Task:**
  - "Sentiment" score capturing how positive/negative d is
- Example scenarios:
  - Infer what people think about a product from text only (i.e., without explicitly given ratings)
  - Historical opinion analysis; e.g., how has people's attitude toward certain politicians changed over time?

Sentiment analysis

- Supervised learning
  - Regression
  - Classification
- Same setup as for document classification:
  - Label a training set with ground-truth sentiment scores
  - Represent documents as features vectors
  - Train supervised model: kNN, linear/logistic regression

Topic detection

Typical task 4

- **Given:**
  - Unlabeled document collections
- **Task:**
  - Determine a set of prevalent topics in the docs
  - Determine for each document to which topics it belongs
- Example scenarios
  - Detection of trending topics in social media (e.g., Twitter)

  - Detection of distinct viewpoints on a political subject
  - Exploratory analysis of a large doc collection

Topic detections

- Clustering
- Represent documents as feature vectors
- Run hierarchical or point-assignment clustering algorithm
  - Hierarchical: agglomerative or divise
  - Point-assignment: e.g., k-means, DBSCAN
- Alternative: matrix factorization (next lecture)

Feature Vectors

Text is not immediately a feature vector: variable length (even for fixed length); positions don't correspond to meaningful features. Text often doesn't have the format for directly running ML algorithms...

- Need to transform arbitrarily long string to fixed-length vector
  - Traditional and vetted: bag of words
  - More recent: *learn* a mapping from strings to vectors (buzzword: "text embedding")

Bag of words

- Bag == multiset
  - E.g., document "what you see is what you get"
    - bag of words {get:1, is:1, see:1, what:2, you:2}
- To have fixed-length representation for all documents:
  - Vector with one entry for each unique word in vocabulary
  - Bag-of-word vectors are very high-dimensional (typically 1e5 or 1e6) and very sparse

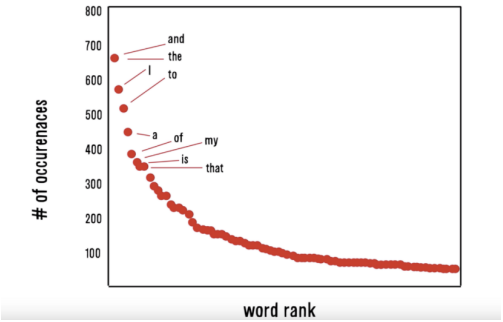
An extra reason for sparsity: Zipf's law

The probability of observing a word scales inversely with its frequency rank:

$$p(w_i) \propto 1/i.$$

(where w_i is the i-th most frequent word)

Word frequency and rank in *Romeo and Juliet* (linear-linear)



we care more about the tail here because they contain more important information

Bag-of-words matrix

- Combine document vectors as rows in a matrix
  - One row per doc
  - One column per word in vocabulary
- This matrix is huge!
  - E.g., Wikipedia: 6M docs, 2M words ⇒ 12 trillion entries
- Solution: use a sparse matrix format
  - Triples: (`doc_idx`, `word_idx`, `count`)
  - E.g., Wikipedia, assuming 2000 words per article on avg.: 12 billion non-zero entries (fits in memory)
- With matrix representation, you're almost ready to use any ML model
  - Be careful when mapping raw text to bag-of-words matrix!
    - Character encoding
    - Language identification
    - Tokenisation
    - Stopword removal
    - Word normalization
  - Tweaking the matrix a bit can lead to much better performance
    - Re-weight/normalize rows and/or columns of matrix

Bag of tricks for bags of words

Character encoding

- Mapping from (abstract) characters to bytes
- Old school: ASCII, Latin-1; new school: Unicode (e.g., UTF-8, UTF-16, UTF-32)
- E.g., W $\Rightarrow$ 0x57 (one byte)
- Reading text from file:
  - Need to read with encoding that was used to write file
  - Especially important for non-English text: à, ê, ü, ...
- Writing to file: Always use UTF-8 or UTF-16; hard-code the output format!
- Otherwise, your future self will be very angry at you

Language identification

- Typically, you're interested in text from a single language
- Increasingly, content is multilingual (e.g., Twitter, Wikipedia)
- Ideally, language code is specified (e.g., headers in HTML; JSON field in Twitter API results)
- But not always... For this, there are good libraries
  - Most commonly based on letter trigrams (e.g., "eau", "ghi", "ijs", "sch", "eiss", "ção", "prv")
  - Much harder if you messed up character encoding...

Tokenization

- Maps character string into sequence of tokens ($\approx$ words)
- E.g., "Hello! How are you?" $\Rightarrow$ `Hello_!_How_are_you_?`
- Tempting to do this yourself by splitting at white spaces and punctuation. But there are many corner cases like emoticons, question and exclamation marks together, ...
- Don't do it yourself, use libraries instead; e.g.,
  - Python: spaCy, nltk; Java; Standford CoreNLP
  - Rule-based, deterministic, fast
- Optimal tokenizer different for different languages, but English tokeniser is often good enough for Latin languages. But in Chinese's there's no white space between words or for **compound works** like in German.

Stopword removal

- Very frequent, "small" words carry little information for most tasks and can "drown out" information contained in real content words
- E.g., "a", "the", "is", "you", "I", punctuation marks
- Many stopword lists online, but be careful!
  - Different tasks require removing different stopwords
  - Good heuristic: remove words appearing in at least $p\%$ of all documents (but what should p be...?)
  - Sometimes stopword removal hurts!
    - Author identification, psychological modeling; punctuation can be useful as well: e.g., "!!!", ":-)"

Word normalization: casefolding

- E.g., "I love yams. Yams are yummy."
- Should "yams" and "Yams"really be different features?
- Simple solution: make everything lower-vase ("casefolding")
- But then "I'd rather have an apple than an Apple.": the phrase loses its sense with casefolding
- Hand-code exceptions? In practice (especially when dataset is large), typically best to **not** do casedfolding
- But when dataset is small, might help because less sparsity.

Word normalization: Stemming

- Map different forms of same word to same, normalized form, by stripping affixes
- E.g., "walking", "walks", "walked" ⇒ "walk"
- Typically done in hacky, heuristic way
- Pros: decreases sparsity in bag-of-words matrix
- Con: discard information
  - E.g., "business" vs. "busy"; "operating" (as in "operating system")
- In English (esp. with big data) typically not done anymore. Still very useful in morphologically richer languages (e.g., Bantu languages)

Word normalization: Lemmatization

- Lemmatization == stemming++
- Map tokens to lexicon entries
- E.g., "U.S.A.", "US" ⇒ "United States"
  "You **lie** in the grass" vs. "You **lie** to me"

- Frequently omitted, as it requires complete lexicon and complex mapping rules
- Especially hard for non-English

Social Media

A real tweet: "ikr smh he asked fir yo last name so he can add u on fb lololol"

Need dedicated toolkits such as **TweenNLP** because of necessary abbreviation, common repeating letters/syllables, ...

Tokens vs. n-grams

- So far: bag-of-words matrix
  - Rows: documents
  - Columns: tokens (a.k.a. unigrams, or 1-grams)
- Frequently, longer sequences belong together
  - E.g., "United States", "operating system"
- Brute-force approach: use $n > 1$
  - E.g., all bigrams (n = 2), all trigrams (n = 3)
  - Using all 5-grams can beat neural networks
  - Problem: combinatorial explosion

Smarter:

- Feature selection ("multi-word expressions", "phrase extraction")
- Simple approach for bigrams: keep bigram if *mutual information* between constituent tokens is large
- How to generalize to $n > 2$?
  - Frequent itemset/sequence mining
  - Wikipedia anchor texts
  - Compressive feature learning

Postprocessing the BOW matrix

Inverse document frequency

- Not all words equally informative. This is the reason for removing stopwords ("a", "the", "is", ...)
- Beyond discarding stopwords, want to give less weight to more common words
  - E.g., "per" vs. "perceptron"

- Standard way: **IDF = inverse document frequency**
  - docfreq(w): number of documents that contain word w
  - N: overall number of documents
  - idf(w) $= -\log(docfreq(w)/N) = \log N - \log(docfreq(w))$
  - Interpretation: information content (in term of bits) of event "randomly drawing a document that contains w"
  - Beyond this theoretical justification, IDF weighting has been show to work well in practice.

TF-IDF matrix

- tf(d, w): term frequency of word w in doc d
  - This is what the bag of words captures
  - E.g., document "what you see is what you get" ⇒ {get:1, is:1, see:1, what:2, you:2}
- idf(w): inverse doc freq of w (computed on entire corpus)
- TF-IDF matrix:
  - Entry in row d and column w has value tf(d, w) · idf(w)
  - Amounts to multiplying column w with constant idf(w)

Row normalization of TF-IDF matrix

- Longer docs have more non-zero entries
- Interpreted as vectors, longer docs have longer vectors
- This may throw off ML algorithms
  - Long vectors far away from short vectors
  - Dot product: random vector has higher dot product with longer vector
- Fix: normalize doc vectors, i.e., rows of TF-IDF matrix
  - L2-normalization: all rows have Euclidean distance 1 from origin (all data points lie on a unit sphere)
  - L1-normalization: all rows sum to 1, i.e., can be interpreted as distribution
- How to know which one is better? **Cross-validation**!

Column normalization

- IDF-scaling may be seen as column normalization
- Additionally, it may help to apply any of the normalization techniques we discussed in lecture 8 ("Applied ML")
  - Min-max scale

  - Standardize: subtract mean; divide by standard deviation

- How to know which one (if any) to use? **Cross-validation**!